Beyond Cycle Count: Sensor-Based Maintenance Warnings for Simulated Turbofan Engines

Prahalad Talur
ptalur09@gmail.com
Eastlake High School
Sammamish, WA, USA

Abstract

Run-to-failure engine data can make a maintenance warning look successful for the wrong reason: elapsed cycle count is already informative about how close an engine is to its recorded failure. This study tests whether measured telemetry adds useful final-window warning information beyond elapsed cycle and current operating conditions. We label a simulated turbofan engine cycle positive when RUL ≤ 20 and use 90 complete trajectories from the public, 100×-downsampled tiny-N-CMAPSS training partition. The corrected input definition uses the 14 measured physical channels documented by N-CMAPSS. It excludes the auxiliary flight-class and health-state fields, unit ID, and RUL.

Five outer engine-held-out folds and four inner engine-grouped folds select a threshold for a 5% pre-window false-alert budget. The cycle-plus-operating baseline recalls 55.4% of final-window cycles. Adding the 14 measured channels raises recall to 79.3% at the matched alert budget. The paired engine-cluster bootstrap estimates a 23.9-percentage-point recall gain, with a 95% interval from 18.3 to 29.6 percentage points.

The result shows that the measured telemetry in this simulator reduces missed final-window cycles beyond a clock-and-operating-state baseline. It does not set a service interval, validate a fielded aircraft system, or certify safety.

Keywords

Predictive maintenance, remaining useful life, turbofan engines, N-CMAPSS, sensor telemetry, warning threshold, engine-grouped cross-validation

ACM Reference Format:

Prahalad Talur. 2026. Beyond Cycle Count: Sensor-Based Maintenance Warnings for Simulated Turbofan Engines. In *Proceedings of International Journal of Secondary Computing and Applications Research (IJSCAR VOL. 3, ISSUE 4)*. ACM, New York, NY, USA, 6 pages. https://doi.org/10.67149/yhjs2024.5/vfmxte3f

1 Introduction

Maintenance teams do not act on a remaining useful life (RUL) estimate alone. They need a warning policy that identifies a near-term maintenance window while keeping alerts during earlier operation under control. That makes the relevant question a classification and threshold-selection problem: does the available information identify cycles near the decision boundary on engines the model has not seen?

N-CMAPSS provides simulated aircraft-engine degradation trajectories under recorded flight conditions [1]. Published work on this dataset has largely reported RUL regression scores with different readers, windows, models, and loss functions [2, 4, 5]. Those RMSE or challenge-score results are not directly comparable to this study, because this study evaluates a binary warning for RUL ≤ 20 and chooses its decision threshold under a false-alert budget.

Run-to-failure data also require a demanding baseline. An engine's elapsed cycle number is associated with its remaining life by construction. If all engines failed at the same cycle, a clock-only rule could solve much of the task. The trajectories used here fail after different lifetimes, so cycle number is informative but not sufficient. A useful sensor result must exceed both the cycle-only and cycle-plus-operating-condition baselines.

This paper first compares four information sets with the same fixed random-forest reference and the same nested engine-level evaluation: elapsed cycle only, cycle plus operating conditions, measured telemetry only, and all observable inputs. It then tests a small candidate set of tabular classifiers inside the outer-training folds to ask whether the warning policy can improve without using test-engine labels. The focus remains information value, not an architecture contest. The claim is narrow: in this simulated setting, the 14 measured channels add final-window warning information beyond elapsed cycle and recorded operating conditions. The experiment does not establish an aircraft maintenance schedule.

2 Data and warning protocol

2.1 Simulated engine data and target

The experiment uses the labeled training partition of the public `tiny-N-CMAPSS` repository at commit `b915997ff8d5` [3]; the reproducibility manifest records the complete commit hash. The source URL is https://github.com/alovberg/tiny-N-CMAPSS. This repository documents a 100×-downsampled N-CMAPSS challenge subset. The checked source files contain 90 simulated run-to-failure trajectories and 633,875 retained readings. We aggregate them to 6,825 engine-cycle rows, or 92.9 retained readings per cycle on average. The separate test partition has no RUL labels and is not used here.

The target marks an engine cycle as inside a stated final maintenance window:

$$y = \begin{cases} 1, & \text{RUL} \leq 20, \\ 0, & \text{RUL} > 20. \end{cases} \tag{1}$$

There are 1,890 positive cycle rows, a 27.7% base rate. The value 20 is an experimental warning boundary, not a recommended service

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC-BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.

IJSCAR VOL. 3, ISSUE 4,
© 2026 The Author(s). Published by IJSCAR.
https://doi.org/10.67149/yhjs2024.5/vfmxte3f

interval. For each trajectory, cycle plus RUL is constant; total simulated lifetime ranges from 48 to 100 cycles (mean 75.8, standard deviation 14.1). This variation makes elapsed cycle a useful but incomplete baseline.

2.2 Feature provenance and input sets

N-CMAPSS distinguishes measured physical properties from auxiliary fields. The primary measurement source used here is the 14-channel `Xs` group. The two fields `Fc` (flight class) and `hs` (health state) are auxiliary data and are excluded from every model. In particular, `hs` is not treated as a sensor measurement. The experiment also excludes unit ID and RUL from every model. Table 1 lists the exact variables.

We compare four inputs with one fixed random-forest reference. The cycle-only baseline has one feature. Cycle plus operating conditions has the cycle number and eight within-cycle operating summaries. Measured telemetry only has 28 within-cycle summaries from the 14 measured channels. All observable inputs has 37 features: cycle, operating summaries, and measured-telemetry summaries. This comparison separates three questions: how much a clock predicts, whether operating state adds information, and whether measured telemetry reduces misses beyond both.

Each variant uses a `RandomForestClassifier` with 250 trees, maximum depth 12, minimum leaf size 3, balanced classes, and a primary random seed of 2026. The model is a fixed screening model. The study does not claim that it is the best architecture for this data.

2.3 Nested engine-held-out validation

We use five outer `GroupKFold` splits grouped by engine. Each outer test fold has 18 complete trajectories and the remaining 72 engines form the outer training data. Every engine appears in exactly one outer test fold. Its rows are absent from fitting and threshold selection for that fold.

Within each outer training set, four inner engine-grouped folds generate out-of-fold probabilities. We choose the lowest probability threshold whose pre-window false-alert rate is at or below 5%. A final model then fits the 72 outer-training engines and predicts the 18 held-out engines. The pooled result contains only these outer-test predictions. This procedure evaluates a warning policy without choosing its threshold from test-engine labels.

2.4 Nested model-family selection

The fixed random forest is used for the information comparison so that the input-set result does not depend on a particular architecture. We also evaluate one secondary, all-observable warning policy that selects a model family within each outer-training split. Its four candidates are the fixed random-forest reference, an extra-trees classifier, a histogram-gradient-boosting classifier, and a standardized RBF support-vector classifier. Their configurations are recorded in the evaluation output.

For each outer fold, every candidate produces inner engine-grouped out-of-fold probabilities. For each candidate, the inner probabilities determine the same 5%-budget threshold as above. The selected candidate is the one with the highest inner final-window recall at that threshold; ties break in order of AUROC, average precision, precision, and lower false-alert rate. We then fit only that selected candidate on all 72 outer-training engines and score the 18 held-out engines. Thus neither model-family selection nor threshold selection uses labels from the outer test engines.

2.5 Metrics, robustness, and warning timing

We report AUROC, precision, final-window recall, and the pre-window false-alert rate. We calculate 95% percentile intervals by resampling complete engine trajectories 2,000 times. We also report a fixed 0.5-threshold check on the same outer-test probabilities. It is a diagnostic, not the policy used for the primary result.

To test stochastic variation, we repeat the full fixed-random-forest input comparison and the full selected-policy procedure with seeds 2024 through 2028. The outer engine splits remain fixed, so these ranges test run-to-run variation on the same data rather than a new-dataset replication.

For a first-warning analysis, let c_u^{window} be the first cycle of engine u with RUL ≤ 20, and let c_u^{warn} be the first cycle whose score meets its outer-fold threshold. The lead is

$$\ell_u = c_u^{\text{window}} - c_u^{\text{warn}}. \tag{2}$$

Positive values are warnings before the window; negative values are warnings after it begins. This timing is a property of the tested screening policy, not a maintenance command.

2.6 Reproducibility materials

The project repository is https://github.com/prahaladtalur/N-CMAPSS-Engine-Prediction. The evaluation records the exact tiny-N-CMAPSS commit, SHA256 hashes of the two downloaded data files, outer-test predictions, selected thresholds, and the scripts used to produce the tables and figures. These materials identify the corrected input definition and make the reported values reproducible.

3 Results

3.1 Measured telemetry adds value beyond cycle and operating state

Table 2 gives the nested outer-test results. Elapsed cycle alone is a strong baseline, with AUROC 0.923 and final-window recall 0.563 at a 0.050 pre-window false-alert rate. Adding operating conditions raises AUROC to 0.935 but leaves recall at 0.554. The all-observable model, which adds summaries from the 14 measured channels, reaches AUROC 0.966 and recall 0.793 at a 0.051 false-alert rate.

The paired outer-test comparison of all observable inputs against cycle plus operating conditions estimates a 0.031 AUROC gain, with a 95% engine-cluster interval from 0.023 to 0.039. Its recall gain is 0.239, with an interval from 0.183 to 0.296. The false-alert-rate difference is 0.004, with an interval that spans zero (-0.006 to 0.015). At the matched alert budget, the added telemetry therefore increases the share of final-window cycles found without a clear increase in the pre-window alert rate.

The telemetry-only rule is also useful without a cycle-number feature. It has AUROC 0.939, precision 0.863, and recall 0.771 at a 0.047 false-alert rate. Compared with cycle only, it gains 0.208 recall (95% interval 0.105 to 0.310). Its AUROC difference is 0.016, with an interval from -0.009 to 0.038. The recall result is the stronger evidence for the fixed 5% warning policy.

Table 1: Input provenance. Means and population standard deviations are computed within each engine cycle for every channel except elapsed cycle.

Group	Count	Fields and use
Measured physical (`Xs`)	14	`T24, T30, T48, T50, P15, P2, P21, P24, Ps30, P40, P50, Nf, Nc, Wf`; included in telemetry variants
Operating conditions (`W`)	4	`alt, Mach, TRA, T2`; used in cycle-plus-operating and all-observable inputs
Elapsed cycle	1	`cycle`; included only in variants that name it
Auxiliary fields	2	`Fc, hs`; excluded from every model
Identifier and label	2	`unit, RUL`; excluded from every model

Table 2: Nested engine-held-out performance at a target 5% pre-window false-alert budget. Values in brackets are 95% percentile intervals from 2,000 complete-engine bootstrap resamples.

Input	AUROC	Precision	Final-window recall	Pre-window false-alert rate
Elapsed cycle only	0.923 [0.910, 0.939]	0.813 [0.766, 0.866]	0.563 [0.480, 0.646]	0.050 [0.032, 0.067]
Cycle plus operating conditions	0.935 [0.922, 0.947]	0.821 [0.775, 0.869]	0.554 [0.480, 0.629]	0.046 [0.031, 0.063]
Measured telemetry only	0.939 [0.918, 0.957]	0.863 [0.837, 0.888]	0.771 [0.715, 0.824]	0.047 [0.037, 0.058]
All observable inputs	0.966 [0.958, 0.973]	0.857 [0.821, 0.892]	0.793 [0.751, 0.835]	0.051 [0.037, 0.066]

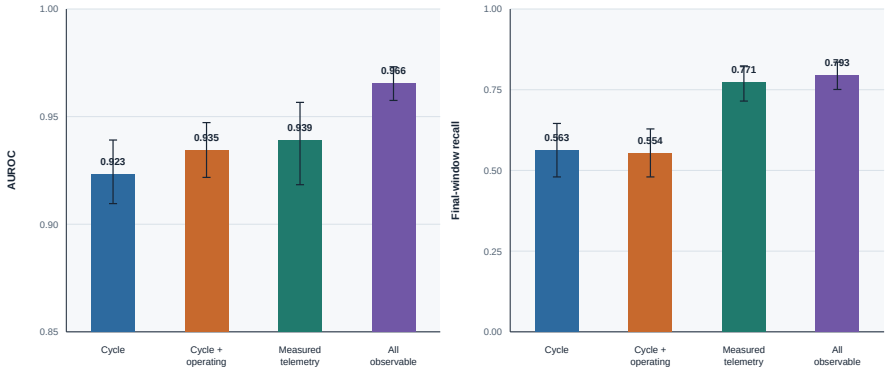


Figure 1: Measured telemetry improves final-window classification beyond elapsed cycle and operating conditions. Each bar uses only outer-test engine predictions; error bars are 95% complete-engine bootstrap intervals.

3.2 Nested selection improves the all-observable warning policy

The information comparison above keeps the classifier fixed. To test whether a stronger warning policy is available without using test-engine labels, the secondary all-observable analysis selects from four tabular candidates inside each outer-training split. Histogram gradient boosting is selected in three outer folds and the RBF support-vector classifier in two. The resulting policy reaches AUROC 0.973, precision 0.861, and final-window recall 0.821 at a 0.051 realized false-alert rate (Table 3). It warns 54 of 90 engines at or before the final-window start, so its gain is in cycle-level detection rather than first-warning lead time.

Relative to the fixed all-observable random forest, nested model selection adds 0.007 AUROC and 0.028 final-window recall at the same pooled false-alert rate. The paired complete-engine bootstrap interval is 0.002 to 0.011 for AUROC and 0.004 to 0.051 for recall. The model-family choice and threshold are made from inner out-of-fold predictions in every outer fold, so these intervals compare outer-test predictions only.

3.3 The fixed 0.5 threshold is a useful diagnostic, not an equivalent policy

Table 4 evaluates the same outer-test scores at a fixed 0.5 threshold. The telemetry-only rule changes little because its nested thresholds range from 0.460 to 0.479. The all-observable, cycle-only, and cycle-plus-operating policies require higher thresholds to meet the 5% alert budget. Their 0.5-threshold recalls are higher, but so are their false-alert rates. This confirms that threshold selection materially

Table 3: All-observable warning policies at a target 5% pre-window false-alert budget. The selected policy chooses its model family and threshold within each outer-training split. Values in brackets are 95% complete-engine bootstrap intervals.

Policy	AUROC	Precision	Final-window recall	Pre-window false-alert rate
Fixed random forest	0.966 [0.958, 0.973]	0.857 [0.821, 0.892]	0.793 [0.751, 0.835]	0.051 [0.037, 0.066]
Nested selected policy	0.973 [0.966, 0.979]	0.861 [0.827, 0.894]	0.821 [0.778, 0.862]	0.051 [0.038, 0.066]

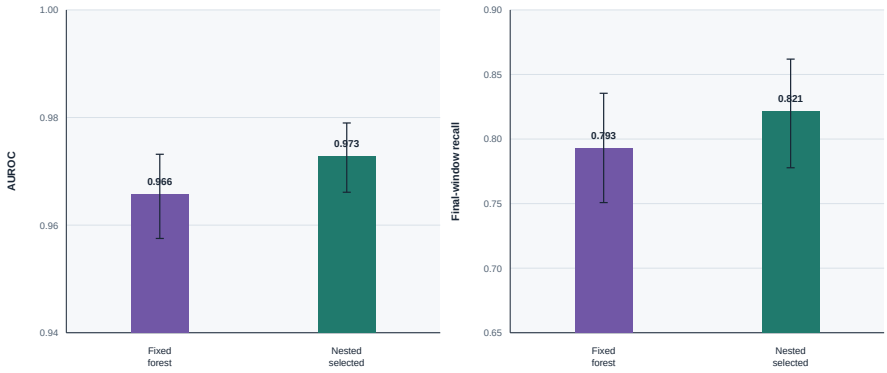


Figure 2: Nested model selection improves the all-observable policy without using labels from test engines. The selected policy uses histogram gradient boosting in three outer folds and an RBF support-vector classifier in two.

affects the decision policy and was not omitted from the primary result.

3.4 Sensor-only scoring produces earlier first warnings

The first-warning analysis evaluates a different operational property from cycle-level recall: whether a policy flags an engine at or before the start of the final 20-cycle window. The telemetry-only rule warns 69 of 90 engines by that point, with a median lead of 6 cycles among the 87 engines it eventually warns (Table 5). The fixed all-observable random forest warns 50 engines by the window start, while the nested selected policy warns 54. Both have a median lead of 1 cycle among the engines they eventually warn.

This difference is a tradeoff, not a contradiction. The all-observable model was selected for cycle-level final-window classification, while the first-alert metric rewards an earlier threshold crossing. A deployment study that needs lead time should select and validate an engine-level early-warning objective directly.

3.5 Cycle-level metrics are stable across model seeds

Table 6 reports five complete nested evaluations with random-forest seeds 2024 through 2028. The seed ranges are narrow for each input. They do not replace external validation, because the simulated engines and the outer folds remain the same.

Across the five seeds, the all-observable recall gain over cycle plus operating conditions ranges from 21.9 to 23.9 percentage points, with a mean of 22.6 points. The paired 18.3-to-29.6-point interval reported for the primary run (seed 2026) captures uncertainty from resampling complete engines; it does not include model-seed variability. First-warning timing is reported for the primary run only and is not part of this seed-robustness check.

4 Discussion

The main result is not that a classifier can recognize a simulated failure trajectory after seeing its age. Cycle count alone already reaches AUROC 0.923. The useful result is the comparison against that strong baseline. At a matched pre-window alert budget, measured telemetry raises final-window recall from 0.554 for cycle plus operating conditions to 0.793 for all observable inputs. The paired complete-engine bootstrap interval for the recall gain remains positive, from 0.183 to 0.296.

The secondary nested model-selection policy raises all-observable recall from 0.793 to 0.821 and AUROC from 0.966 to 0.973. It selects histogram gradient boosting in three outer folds and an RBF support-vector classifier in two. This is a practical improvement under the same warning budget, but it is not an architecture claim. The candidate set is small, the model choice is evaluated only on this simulated subset, and the policy does not improve the earlier first-warning timing reported for the sensor-only rule.

The telemetry-only rule supplies a separate result for an early-screening setting. It has no elapsed-cycle feature, yet it recalls 77.1% of final-window cycles at a 4.7% pre-window false-alert rate. It warns 69 of 90 engines by the window start, with a median lead of 6 cycles among warned engines. This does not show that the

Table 4: Diagnostic fixed-threshold performance on the same outer-test probabilities. These values do not replace nested threshold selection.

Input	Precision	Final-window recall	Pre-window false-alert rate
Elapsed cycle only	0.624	0.887	0.204
Cycle plus operating conditions	0.700	0.859	0.141
Measured telemetry only	0.890	0.751	0.036
All observable inputs	0.791	0.872	0.089

Table 5: First-warning timing from outer-test predictions. Positive lead means the first warning occurs before the final-window start.

Input	By window start	Late warning	No warning	Median lead among warned
Measured telemetry only	69/90 (76.7%)	18/90	3/90	6 cycles
All observable, fixed random forest	50/90 (55.6%)	39/90	1/90	1 cycle
All observable, nested selected policy	54/90 (60.0%)	34/90	2/90	1 cycle



Figure 3: First warning for the telemetry-only policy across 90 outer-test engine trajectories. This policy uses no cycle number or operating-condition field. Positive lead values arrive before the final 20-cycle window begins.

Table 6: Five-seed robustness check. Each entry is mean [minimum, maximum] over full nested evaluations using fixed outer engine splits. The first four rows use the fixed random-forest reference; the final row repeats model-family selection inside every outer-training split.

Input	AUROC	Final-window recall	Pre-window false-alert rate
Elapsed cycle only	0.923 [0.923, 0.923]	0.564 [0.559, 0.568]	0.050 [0.048, 0.051]
Cycle plus operating conditions	0.935 [0.934, 0.935]	0.559 [0.554, 0.566]	0.049 [0.046, 0.049]
Measured telemetry only	0.940 [0.939, 0.941]	0.774 [0.771, 0.776]	0.048 [0.046, 0.050]
All observable inputs	0.965 [0.965, 0.966]	0.785 [0.780, 0.793]	0.050 [0.049, 0.051]
Nested selected policy	0.973 [0.973, 0.973]	0.821 [0.821, 0.822]	0.051 [0.051, 0.051]

model knows when a real engine should be removed from service. It shows that the documented measured channels contain signal about the simulated approach to the chosen 20-cycle boundary.

The feature definition matters. The evaluated model uses only the 14 named measured physical channels, listed in Table 1. It does not use virtual sensors, target health parameters, unit ID, RUL, or the auxiliary `Fc` and `hs` fields. This restriction makes the telemetry result testable. It also makes the result lower than an experiment that treated auxiliary health-state information as if it were a measured sensor.

Several limitations remain. First, tiny-N-CMAPSS is simulated and the retained data are 100× downsampled. Each per-cycle summary averages about 93 retained readings, so it cannot test how a warning would respond to high-rate within-flight transients. Second, the 20-cycle window, 5% false-alert budget, and one-cycle alert definition are experimental choices. A maintenance program would need costs, inspection capacity, confirmation rules, and a lead-time objective before choosing a policy. Third, five seeds test stochastic variation of the fixed reference and the nested selection procedure on the same data. They do not replace an independently labeled fleet or a held-out test campaign.

The next technical step is to train and evaluate a warning policy against an engine-level objective, such as warning by a specified lead time with a per-engine alert budget. That would directly optimize the tradeoff exposed in Table 5. The next empirical step is external validation on labeled engines with maintenance outcomes. Until then, this paper should be read as a controlled simulation result about information value, not a deployable maintenance recommendation.

5 Conclusion

This study tests a narrow maintenance-warning question on simulated turbofan engines: do measured channels add information beyond elapsed cycle and operating conditions when the warning threshold is chosen without seeing the held-out engines? The answer is yes for the tested N-CMAPSS subset. At a roughly 5% pre-window false-alert rate, all observable inputs raise final-window recall from 55.4% to 79.3% over the cycle-plus-operating baseline. The paired engine-cluster interval for the gain is 18.3 to 29.6 percentage points.

The telemetry-only result is also useful. With no cycle number or operating-condition field, it recalls 77.1% of final-window cycles and warns 69 of 90 engines by the window start in the primary run (seed 2026). Its cycle-level recall is stable across five random-forest seeds, ranging from 77.1% to 77.6%; first-warning timing was not repeated across seeds. These results use only the 14 documented measured channels. A secondary policy that selects its model family inside the training folds raises the all-observable result to AUROC 0.973 and recall 82.1%, compared with 0.966 and 79.3% for the fixed random forest at the same pooled false-alert rate. This is a controlled performance gain, not a claim of a universally best model.

The contribution is evidence that observed sensor telemetry carries maintenance-warning value beyond a flight-cycle clock in this simulator. It is not evidence for a service interval or a fielded-aircraft decision rule. Those claims require maintenance-defined costs and lead times, plus independent labeled engine data.

References

[1] Manuel Arias Chao, Chetan Kulkarni, Kai Goebel, and Olga Fink. 2021. Aircraft Engine Run-to-Failure Dataset under Real Flight Conditions for Prognostics and Diagnostics. *Data* 6, 1 (2021), 5. doi:10.3390/data6010005

[2] Manuel Arias Chao, Chetan Kulkarni, Kai Goebel, and Olga Fink. 2022. Fusing physics-based and deep learning models for prognostics. *Reliability Engineering & System Safety* 217 (2022), 107961. doi:10.1016/j.ress.2021.107961

[3] Andreas Lövberg. [n. d.]. tiny-N-CMAPSS: Downsampled N-CMAPSS challenge data. GitHub repository, https://github.com/alovberg/tiny-N-CMAPSS. Pinned commit: https://github.com/alovberg/tiny-N-CMAPSS/tree/b915997ff8d571f4e9d091954d6556835f212ded; accessed 2026-08-12. https://github.com/alovberg/tiny-N-CMAPSS

[4] Andreas Lövberg. 2021. Remaining Useful Life Prediction of Aircraft Engines with Variable Length Input Sequences. *Annual Conference of the PHM Society* 13, 1 (2021). doi:10.36001/phmconf.2021.v13i1.3108

[5] David Solís-Martín, Juan Galán-Páez, and Joaquín Borrego-Díaz. 2021. A Stacked Deep Convolutional Neural Network to Predict the Remaining Useful Life of a Turbofan Engine. *Annual Conference of the PHM Society* 13, 1 (2021). doi:10.36001/phmconf.2021.v13i1.3110